



PROJETO E ANÁLISE DE ALGORITMOS

AULA 1 - INTRODUÇÃO

BIBLIOGRAFIA

- CORMEN, Thomas H.; LEISERSON, Charles E.; Ronald L. Rivest; et al. Algoritmos. Rio de Janeiro: Grupo GEN, 2024.
- KLEINBERG, J., TARDOS, E.; Algorithm Design, Addison-Wesley, 2005.
- SKIENA, S. S., The algorithm design manual, Springer, 2010.

CONTEÚDO

- Revisão de álgebra.
- Relações de recorrência.
- Análise de algoritmos através de funções assintóticas, incluindo notação Big-O e demais funções assintóticas.
- Análise de problemas NP-completo e reducibilidade.
- Aplicação da análise assintótica na avaliação de algoritmos de ordenação.
- Análise e projeto de algoritmos de busca.

CONTEÚDO

- Algoritmos randômicos.
- Algoritmos baseados em abordagem gulosa.
- Algoritmos baseados e programação dinâmica.
- Algoritmos para grafos: caminho mínimo, árvore geradora, detecção de ciclos, etc

AVALIAÇÃO

- Avaliação: $([P1 + T1] + [P2 + T2])/2$
- *Trabalho 1 e 2 somam até dois pontos em suas respectivas provas (não há pesos nas provas).

1 ANÁLISE DE ALGORITMOS

- Analisar um algoritmo significa prever os recursos de que o algoritmo necessita. Podemos considerar recursos como memória, largura de banda de comunicação ou consumo de energia.
- Em geral, pela análise de vários algoritmos candidatos para um problema, pode-se identificar facilmente um que seja o mais eficiente.
- Essa análise pode indicar mais de um candidato viável, porém, normalmente, podemos descartar vários algoritmos de qualidade inferior no processo.

1.1 ALGORITMOS

- Definição de Algoritmo: informalmente, um algoritmo é qualquer procedimento computacional bem definido que toma algum valor ou conjunto de valores como entrada e produz algum valor ou conjunto de valores como saída. Portanto, um algoritmo é uma sequência de etapas computacionais que transformam a entrada na saída.
- Também podemos considerar um algoritmo como uma ferramenta para resolver um problema computacional bem definido. O enunciado do problema especifica em termos gerais a relação desejada entre entrada e saída. O algoritmo descreve um procedimento computacional específico para se conseguir essa **relação entre entrada e saída**.

1.2 O PROBLEMA DA ORDENAÇÃO

- Considere que o nosso objetivo seja ordenar uma sequência de números em ordem não decrescente.
- O problema de ordenação pode ser **formalmente** definido da seguinte forma:
- Entrada: Uma sequência de n números $\langle a_1, a_2, \dots, a_n \rangle$.
- Saída: Uma permutação (reordenação) $\langle a'_1, a'_2, \dots, a'_n \rangle$ da sequência de entrada, tal que $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

1.2 O PROBLEMA DA ORDENAÇÃO

- Por exemplo, dada a sequência de entrada $\langle 31, 41, 59, 26, 41, 58 \rangle$, um algoritmo de ordenação devolve como saída a sequência $\langle 26, 31, 41, 41, 58, 59 \rangle$.
- Tal sequência de entrada é denominada **instância** do problema de ordenação.
- Em geral, uma instância de um problema consiste na entrada (que satisfaz quaisquer restrições impostas no enunciado do problema) necessária para calcular uma solução para o problema.

1.2 O PROBLEMA DA ORDENAÇÃO

- Diz-se que um algoritmo é **correto** se, para toda instância de entrada, ele parar com a saída correta. Dizemos que um algoritmo correto resolve o problema computacional dado.
- Um algoritmo incorreto poderia não parar em algumas instâncias de entrada ou poderia parar com uma resposta incorreta.
- Ao contrário do que se poderia esperar, às vezes os algoritmos incorretos podem ser **úteis**, se pudermos controlar sua taxa de erros.

1.3 EFICIÊNCIA

- Embora os computadores sejam cada vez mais rápidos e possuam cada vez mais memória, esses **recursos são limitados**. É necessário pensar em algoritmos que sejam eficientes em termos de tempo ou espaço.
- Algoritmos diferentes criados para resolver o mesmo problema muitas vezes são muito diferentes em termos de eficiência. Essas diferenças podem ser muito mais significativas que as diferenças relativas a hardware e software.

1.3 EFICIÊNCIA

- O problema da ordenação também é um bom exemplo do ponto de vista de eficiência:
- Realmente faz diferença usar o quicksort ao invés do bubblesort para qualquer sequência numérica?
- O tamanho da entrada é importante nessa comparação?
- A análise de algoritmos nos fornece ferramentas para que se possa fazer essa comparação de forma satisfatória.

1.3 EFICIÊNCIA

- Buscamos algoritmos que sejam **corretos e eficientes**, ao mesmo tempo em que sejam **fáceis de implementar**. Essas metas podem não ser simultaneamente atingíveis.
- Em ambientes industriais, qualquer programa que pareça dar respostas boas o suficiente sem tornar o aplicativo mais lento é frequentemente aceitável, independentemente de existir um algoritmo melhor.

1.3 EFICIÊNCIA

- Considere o seguinte exemplo envolvendo dois algoritmos de ordenação:
- O primeiro, conhecido como ordenação por inserção, leva um tempo aproximadamente igual a $c_1 n^2$ para ordenar n itens, em que c_1 é uma constante que não depende de n . Isto é, ele demora um tempo aproximadamente proporcional a n^2 .
- O segundo, de ordenação por intercalação, leva um tempo aproximadamente igual a $c_2 n \lg(n)$, em que $\lg(n)$ representa $\log_2(n)$ e c_2 é outra constante que também não depende de n .

1.3 EFICIÊNCIA

- Considere ainda dois computadores, um computador mais rápido (computador A) que executa a ordenação por inserção com um computador mais lento (computador B) que executa a ordenação por intercalação. Cada um deve ordenar um vetor de 10 milhões de números.
- Suponha que o computador A execute 10 bilhões de instruções por segundo e que o computador B execute apenas 10 milhões de instruções por segundo; assim, o computador A será 1.000 vezes mais rápido que o computador B em capacidade bruta de computação.

1.3 EFICIÊNCIA

- Para tornar a diferença ainda mais drástica, suponha que o programador mais astucioso do mundo codifique a ordenação por inserção em linguagem de máquina para o computador A e que o código resultante exija $2n^2$ instruções para ordenar n números.
- Suponha ainda que um programador médio implemente a ordenação por intercalação utilizando uma linguagem de alto nível com um compilador ineficiente, sendo que o código resultante totaliza $50n\lg(n)$ instruções.

1.3 EFICIÊNCIA

- Para ordenar 10 milhões de números, o computador A leva:
- $\frac{2 \cdot (10^7)^2 \text{ instruções}}{10^{10} \text{ instruções/segundo}} = 20000 \text{ segundos (mais de 5,5 horas)}$
- Por outro lado, o computador B leva
- $\frac{5 \cdot 10^7 \lg 10^7 \text{ instruções}}{10^7 \text{ instruções/segundo}} \approx 1163 \text{ segundos (menos de 20 minutos)}$

1.3 EFICIÊNCIA

- Usando um algoritmo cujo tempo de execução cresce mais lentamente, até mesmo com um compilador fraco, o computador B executa numa rapidez mais de 17 vezes maior que o computador A!
- A vantagem da ordenação por intercalação é ainda mais evidente quando ordenamos 100 milhões de números: a ordenação por inserção demora mais de 23 dias, enquanto a ordenação por intercalação demora menos de quatro horas.

1.3 EFICIÊNCIA

- O exemplo anterior mostra que algoritmos, assim como o hardware de computadores, devem ser considerados uma tecnologia. O desempenho total do sistema depende da escolha de algoritmos eficientes tanto quanto da escolha de hardware rápido. A velocidade dos avanços que ocorrem em outras tecnologias computacionais também é observada em algoritmos.
- **Uma sólida base de conhecimento e técnica algorítmica é uma das características que definem o programador verdadeiramente qualificado.**

1.4 MEDIDAS DE COMPLEXIDADE

- A **quantidade de trabalho** requerido por um algoritmo não pode ser descrita simplesmente por um número, porque o número de operações básicas efetuadas em geral não é o mesmo para qualquer entrada (depende do tamanho da entrada).
- Por exemplo, numa inversão de matrizes, o número de operações (de adição e multiplicação) varia com a dimensão da matriz, ou na ordenação de uma lista, varia com o número de elementos da lista. Mesmo para entradas do mesmo tamanho, o número de operações efetuadas pelo algoritmo pode depender de uma entrada particular.

1.4 MEDIDAS DE COMPLEXIDADE

- Para ordenar uma lista quase classificada, pode não ser necessário o mesmo esforço que para ordenar uma lista de mesmo tamanho, mas com os elementos em grande desordem. A expressão "quantidade de trabalho requerido" também é chamada complexidade do algoritmo.
- A complexidade também pode ser vista como uma **propriedade do problema**. Nesse caso, é possível obter uma medida independente do tratamento dado ao problema ou do caminho percorrido na busca da solução, portanto independente do algoritmo.

1.4 MEDIDAS DE COMPLEXIDADE

- Se a complexidade é tomada como a **máxima** para qualquer entrada de um dado tamanho, a complexidade é chamada complexidade no pior caso ou simplesmente complexidade. Se, entretanto, é levada em conta a probabilidade de ocorrência de cada entrada de um mesmo "tamanho", a complexidade é chamada complexidade esperada ou complexidade média.
- Temos, na verdade, que distinguir **três cenários**: melhor caso, pior caso e caso médio. O melhor caso corresponde ao menor tempo de execução sobre todas as possíveis entradas de tamanho n .

1.4 MEDIDAS DE COMPLEXIDADE

- O pior caso corresponde ao maior tempo de execução sobre todas as entradas de tamanho n . Se f é uma função de complexidade baseada na análise de pior caso, então o **custo** de aplicar o algoritmo nunca é maior do que $f(n)$.
- O caso médio (ou caso esperado) corresponde à média dos tempos de execução de todas as entradas de tamanho n . Na análise do caso esperado, uma distribuição de probabilidades sobre o conjunto de entradas de tamanho n é suposta, e o custo médio é obtido com base nessa distribuição.

1.4.1 COMPLEXIDADE DE TEMPO

- Uma das medidas de complexidade **mais importante** é a complexidade de tempo. Existem vários aspectos a considerar na determinação da complexidade de tempo de um algoritmo.
- Um critério pode ser medir o tempo de execução requerido por diferentes algoritmos para a solução de um problema particular em um computador específico.
- Entretanto, uma medida empírica é fortemente **dependente**, tanto do programa quanto da máquina usada para implementar o algoritmo.

1.4.1 COMPLEXIDADE DE TEMPO

- Assim, uma pequena mudança no programa pode não representar uma mudança significativa no algoritmo, mas, apesar disso, a velocidade de execução pode ser afetada. Além disso, se dois programas são comparados, primeiro numa máquina, depois em outra, as comparações podem dar resultados diferentes.
- Assim, apesar de a comparação de programas reais, rodando em computadores reais, ser uma fonte de informação importante, os resultados são inevitavelmente **afetados** pela programação e pelas **características das máquinas**.

1.4.1 COMPLEXIDADE DE TEMPO

- Uma alternativa útil é uma análise matemática das **dificuldades intrínsecas** da resolução do problema computacionalmente.
- E essa análise permite, muitas vezes, antecipar o cálculo da complexidade para a fase de projeto do algoritmo, influenciando o resultado final.

1.4.1 COMPLEXIDADE DE ESPAÇO

- A memória usada por um programa, bem como o tempo requerido para executar um programa, depende da implementação particular. Entretanto, algumas conclusões sobre espaço utilizado podem ser tiradas, examinando-se o algoritmo.
- Um programa requer uma área para guardar suas instruções, suas constantes, suas variáveis e os dados. Pode também utilizar uma área de trabalho para manipular os dados e guardar informações para levar adiante a computação.

1.4.1 COMPLEXIDADE DE ESPAÇO

- Os dados podem ser representados de várias formas, as quais requerem uma área maior ou menor.
- Se os dados têm uma representação natural, por exemplo, como uma matriz, são considerados para análise de espaço somente o espaço extra, além do espaço utilizado para guardar as instruções do programa e os dados.
- Porém, se os dados podem ser representados de várias formas, como um grafo, o espaço requerido para guardá-los também deve ser levado em conta.

1.4.3 CRITÉRIOS DE COMPLEXIDADE

- Para medir a quantidade de trabalho realizado por um algoritmo, é escolhida uma operação, chamada operação fundamental, e então é contado o número de execuções dessa operação na execução do algoritmo.
- A operação escolhida como fundamental deve ser tal que a contagem do número de vezes que ela é executada expresse a quantidade de trabalho do algoritmo, dispensando outras medidas.
- Às vezes, é necessário mais de uma operação fundamental, com pesos diferentes.

1.4.3 CRITÉRIOS DE COMPLEXIDADE

- Por exemplo, para um algoritmo de ordenação, uma operação fundamental natural é a comparação entre elementos quanto à ordem.
- Para o caso da busca em tabela, uma operação fundamental razoável consiste na comparação, quanto à igualdade, de um elemento da tabela com a chave buscada.

1.5 O MODELO RAM

- Antes de analisar um algoritmo, devemos ter um modelo da tecnologia em que ele será usado, inclusive um modelo para os recursos dessa tecnologia e uma forma de expressar seus custos.
- Vamos considerar um modelo de computação genérico de máquina de acesso aleatório (RAM, do inglês random-access machine) com um único processador como nossa tecnologia de implementação e entenderemos que nossos algoritmos serão implementados como programas de computador.

1.5 O MODELO RAM

- No modelo de RAM, as instruções são executadas uma após outra, sem operações concorrentes.
- O modelo de RAM assume que cada instrução leva a mesma quantidade de tempo que qualquer outra instrução e que cada acesso aos dados — usando o valor de uma variável ou armazenando em uma variável — leva a mesma quantidade de tempo que qualquer outro acesso a dados.
- Em outras palavras, no modelo de RAM, cada instrução ou acesso a dados leva um tempo constante — até mesmo indexação em um vetor.

1.5 O MODELO RAM

- No sentido estrito, deveríamos definir com precisão as instruções do modelo de RAM e seus custos.
- Porém, isso seria tedioso e nos daria pouca percepção do projeto e da análise de algoritmos.
- O modelo de RAM contém instruções comumente encontradas em computadores reais: instruções aritméticas (como soma, subtração, multiplicação, divisão, resto, piso, teto), de movimentação de dados (carregar, armazenar, copiar) e de controle (desvio condicional e incondicional, chamada e retorno de sub-rotinas).

1.5 O MODELO RAM

- Os tipos de dados no modelo de RAM são inteiros, de ponto flutuante e de caracteres. No modelo de RAM, não tentamos modelar a hierarquia da memória que é comum em computadores contemporâneos. Isto é, não modelamos caches ou memória virtual.